



UNCUYO
UNIVERSIDAD
NACIONAL DE CUYO



**FACULTAD
DE INGENIERÍA**

Ingeniería del Software II

INFORME DE PROYECTO INTEGRADOR

PROYECTO INTEGRADOR N°1: “Zero Shop”



EQUIPO DE TRABAJO:

Grupo: Knuth-Morris-Pratt

Integrantes:

- Mateo Peralta – Legajo: 14660
- Luciano Cánovas – Legajo: 14324
- Juan Cruz Ambrosini – Legajo: 11906

DOCENTES A CARGO:

- Lucía Cortes
- Leandro Spadaro

AÑO LECTIVO:

- 2026



[1. Metodología de desarrollo.](#)

[2. Proceso de desarrollo.](#)

[2.a Introducción al proyecto.](#)

[Especificación del proyecto.](#)

[2.b Fase de inicialización.](#)

[Visión del Proyecto.](#)

[Entrevista con el cliente \(Simulada\).](#)

[Objetivos y Alcance del Sistema.](#)

[Actores del Sistema.](#)

[Requisitos No Funcionales.](#)

[2.c Fase de elaboración.](#)

[Requisitos Funcionales.](#)

[Análisis y Diseño.](#)

[Patrones de Arquitectura y Diseño Aplicados.](#)

[2.d Fase de construcción.](#)

[Flujo de Trabajo de Implementación.](#)

[Testing \(Pruebas unitarias y de interfaz\).](#)

[Gestión del Cambio](#)

[2.e Fase de transición y despliegue.](#)

[3. Propuesta de mejora para software de gimnasios.](#)

[3.a Referencias de mercado](#)

[3.b Propuestas de mejora](#)

[4. Propuestas de refactorización](#)

[4.a Propuestas de refactorización de la vista](#)

[4.b Propuestas de refactorización de los controladores](#)

[4.c Propuestas de refactorización del modelo](#)

[4.d Propuestas de refactorización del acceso a datos](#)

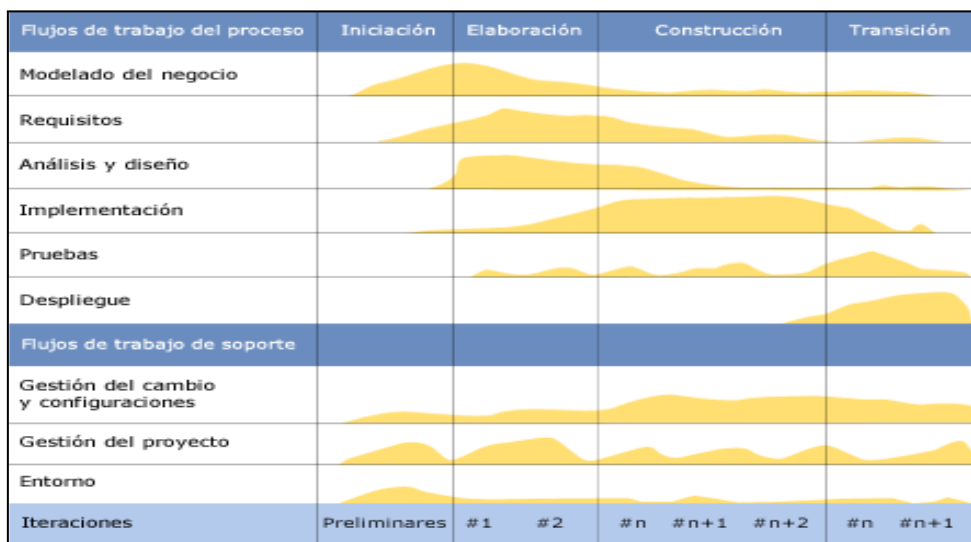
[5. Material de exposición en conferencia](#)

[6. Referencias](#)

[7. Anexo](#)

1. Metodología de desarrollo.

La ejecución del trabajo se fundamenta en la metodología del Proceso Unificado de Rational (RUP), aplicando un enfoque iterativo para el abordaje de sus etapas y flujos de trabajo correspondientes.



2. Proceso de desarrollo.

2.a Introducción al proyecto.

Especificación del proyecto.

La plataforma web se encuentra implementada integralmente mediante el lenguaje Java, utilizando Bootstrap sobre plantillas HTML para la elaboración del frontend. El entorno de desarrollo integrado (IDE) seleccionado por el equipo comprende las herramientas IntelliJ IDEA y VS Code, apoyándose en Apache Tomcat como servidor de aplicaciones para la ejecución de Spring. Respecto al almacenamiento persistente, se adoptó PostgreSQL administrado vía pgAdmin, empleando el mapeo objeto-relacional a través de las anotaciones de JPA. El control de versiones y la gestión de código se llevaron a cabo utilizando la plataforma GitHub, disponible en el siguiente repositorio: <https://github.com/lucianocanovas/zero-shop.git>.

La coordinación del equipo de trabajo se articuló mediante instancias presenciales complementadas con canales de comunicación remota como Discord y WhatsApp. Para la elicitación, especificación y asignación de requerimientos, se empleó la herramienta Trello en conjunto con mensajería instantánea.

Para la elaboración del modelado de artefactos y diagramas, se emplearon Draw.io y Mermaid, mientras que el diseño de los prototipos de interfaz se realizó en Figma. La verificación y



validación del correcto funcionamiento del software se llevaron a cabo mediante pruebas unitarias implementadas con JUnit y Mockito.

El sistema desarrollado consiste en un portal de comercio electrónico orientado al rubro de indumentaria deportiva para una firma ubicada en la provincia de Mendoza.

2.b Fase de inicialización.

Visión del Proyecto.

El objetivo principal del sistema Zero Shop es suministrar un entorno transaccional para clientes finales y un módulo de administración interna para el negocio. La solución contempla la clasificación de productos por segmentos (damas, caballeros y niños, con sus correspondientes divisiones en calzado, ropa y accesorios) junto con la exhibición detallada de catálogos. Asimismo, abarca el ciclo integral del comercio electrónico, incluyendo el procesamiento de pagos, la logística de envíos y la generación de informes de gestión.

Entrevista con el cliente (Simulada).

- ¿Todos los productos actualizarán su precio al mismo tiempo o de forma individual?
- + Debido a la inflación en Argentina, necesitamos una funcionalidad que nos permita actualizar el precio de los productos de forma general cada dos meses.
- ¿Es obligatorio que los clientes tengan correo electrónico para comprar?
- + Sí, es totalmente obligatorio. Vamos a enviar un código al correo para que activen su cuenta, y también lo usaremos para enviarles las facturas de compra y un newsletter de ofertas cada 10 días.
- ¿Cómo desean comunicarse con los proveedores cuando se quedan sin stock?
- + Lo más práctico para nosotros es usar WhatsApp. Queremos que el sistema nos muestre cuándo el stock es "Malo" (menor al 20%) y nos habilite un botón para mandarles un mensaje directo para pedir reposición.
- ¿El sistema sólo aceptará pagos online con tarjeta?
- + No, además de la integración principal con Mercado Pago para las compras web, el sistema debe contemplar el registro de pagos en efectivo.
- ¿Habrá usuarios con privilegios especiales en la administración?
- + Sí, el sistema debe tener roles estrictos. El Usuario Cliente solo puede comprar y ver su perfil, Los usuarios Empleados deben poder acceder a la información del negocio y acceder a un punto de venta para la venta presencial en las sucursales mientras que el Usuario Administrador es el único que puede cambiar precios, agregar proveedores y modificar el stock.

Objetivos y Alcance del Sistema.

El sistema fue diseñado para cumplir con las siguientes funcionalidades enfatizadas por el cliente:



- Administrar compras mediante un carrito interactivo, integrando múltiples medios de pago (Efectivo, Mercado Pago) y enviando comprobantes automáticos por correo electrónico.
- Registrar el aumento de stock mediante la recepción de órdenes de compra a proveedores, y su disminución automática al realizar el pago de una venta.
- Actualización automática de precios cada dos meses en respuesta a la inflación.
- Proveer al personal administrativo informes de:
 - Ventas: Filtradas por rango de fechas detallando totales, productos y formas de pago.
 - Stock: Semaforización del stock por sucursales (Bien >50%, Regular 20-50%, Malo <20%) con integración directa a WhatsApp para solicitar reposición a proveedores.
 - Proveedores: Comparativa de costos para asegurar la compra más económica.
- Envío automático de newsletters cada 10 días al correo electrónico destacando los productos en oferta.

Actores del Sistema.

El uso del sistema se divide estrictamente en 3 roles:

1. Cliente: Navega por el catálogo público. Debe registrarse y completar su perfil con la información necesaria para contactarla y realizar el envío. Puede realizar compras, anularlas, pagarlas, hacer seguimiento de las entregas y modificar su información personal.
2. Empleado: Tiene acceso al dashboard del sistema en modo lectura, puede acceder al punto de venta de la sucursal para vender presencialmente.
3. Administrador: Tiene acceso al dashboard del sistema. Sus responsabilidades incluyen el ABM de usuarios, productos, precios, sucursales, y proveedores. Además, se encarga de generar órdenes de compra a proveedores y actualizar el stock cuando sea necesario.

Requisitos No Funcionales.

Los siguientes requerimientos no funcionales se establecen según lo solicitado por el cliente en la entrevista:

- El sistema debe estar desarrollado en su totalidad en el lenguaje Java utilizando el framework Spring Boot junto con JPA para la persistencia de datos en el motor PostgreSQL.
- Las contraseñas de los usuarios deben persistir de forma fuertemente encriptada. Se requiere validación asíncrona de identidad vía email.

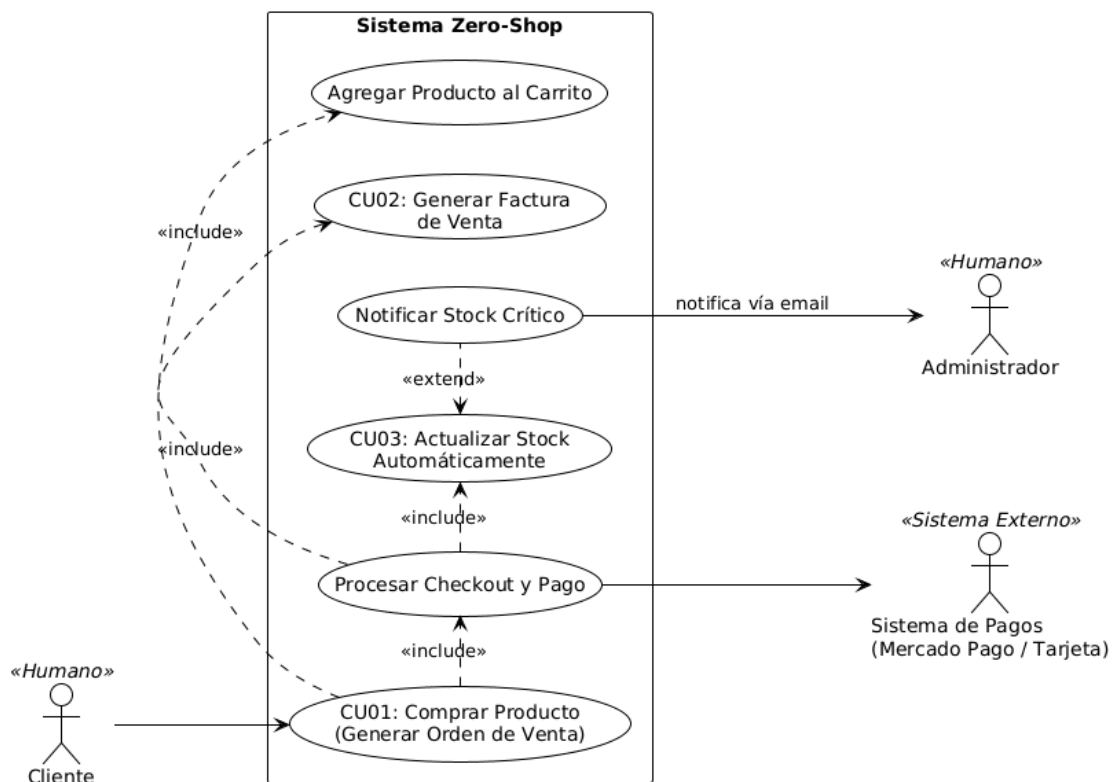
- La interfaz institucional y administrativa debe resolverse íntegramente mediante HTML/CSS, utilizando el framework Bootstrap integrado con plantillas Thymeleaf, lo que garantiza un diseño limpio, responsivo y profesional, sin mucho costo.
- Integraciones Externas: El sistema debe gestionar las conexiones externas utilizando exclusivamente SDKs, módulos nativos de Spring y redirecciones. Se utilizara el SDK oficial de Mercado Pago (una librería Java) para delegar el proceso de cobro sin manipular la red ni usar APIs, se emplea el módulo interno JavaMailSender para la mensajería de correos y se utiliza Deep Linking (wa.me) directamente en el frontend para abrir WhatsApp, manteniendo el backend de Java completamente libre de conexiones y configuraciones de API.

2.c Fase de elaboración.

Una vez delimitados los requerimientos e imponderables definidos previamente, se procede con la especificación de la arquitectura del sistema, el esquema de datos relacional y el diseño detallado de las funcionalidades críticas para la solución Zero Shop, garantizando la solvencia técnica previa a la etapa de desarrollo.

Requisitos Funcionales.

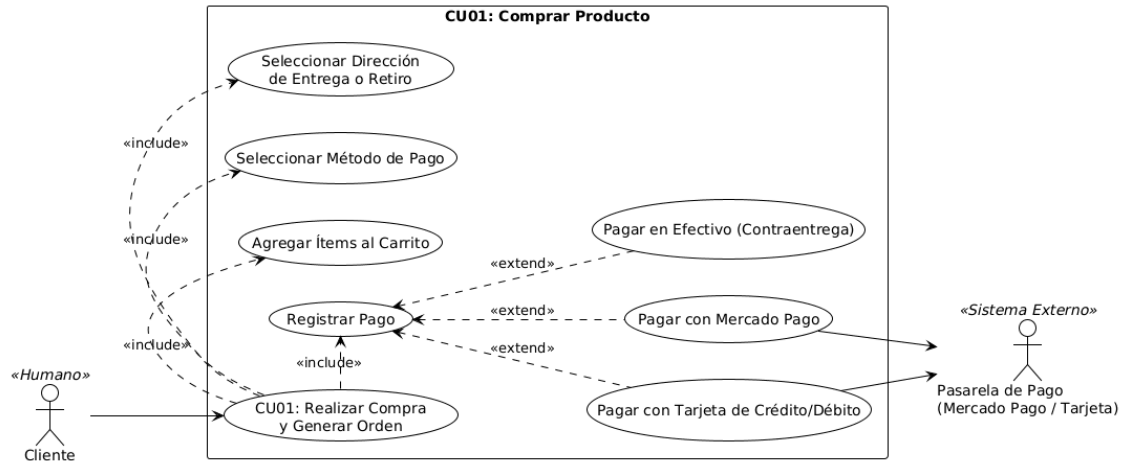
Diagrama de Casos de Uso: Para consultar la representación completa de los modelos elaborados.



ver el [Diagrama de caso de uso - Casos de uso críticos en el Anexo.](#)

Escenarios de Casos de Uso Críticos

Escenario: Generar Orden de Compra.



Ver la especificación detallada del [Escenario de caso de uso - Orden de compra cliente](#) y su correspondiente [Diagrama de secuencia - Orden de compra de cliente.](#)

a. Historia de usuario:

- i. ID: US-ORD-01
- ii. Título: Creación y procesamiento de una orden de compra de cliente.
- iii. Cómo: Cliente
- iv. Quiero: Poder confirmar mi carrito de compras seleccionando la sucursal de retiro/despacho, la dirección de envío y método de pago.
- v. Para: Concretar la adquisición de los productos seleccionados y recibir el seguimiento de mi pedido.

vi. Criterios de aceptación:

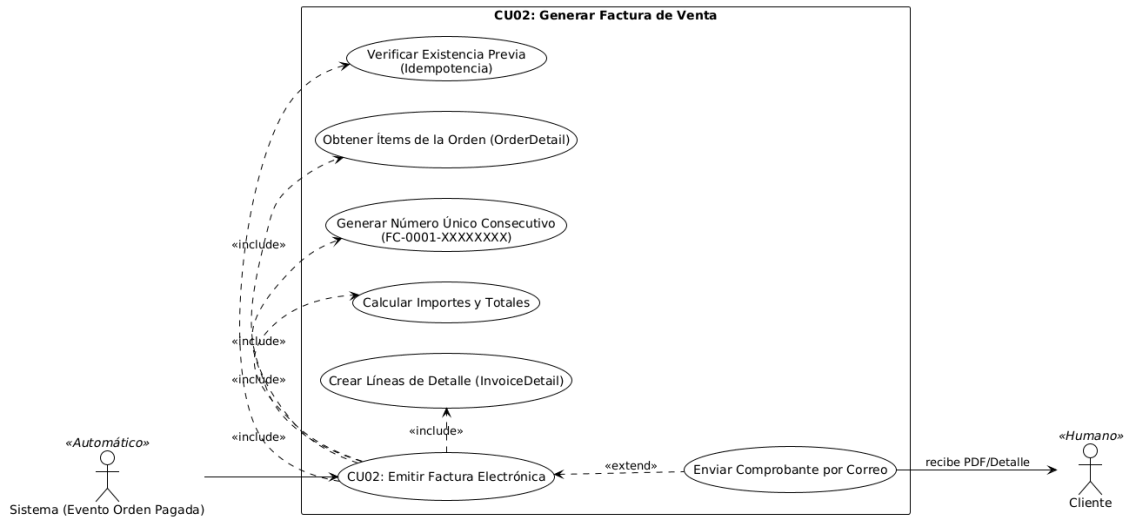
1. Dado que el cliente tiene artículos en su carrito (ON_CART) y existe disponibilidad en la sucursal elegida, cuando procede al checkout y selecciona el método de pago, entonces el estado de la orden transiciona a PENDING_PAYMENT o PAID.
2. Dado que se confirma el pago (PAID), cuando se procesa la transacción, entonces el stock de los productos es descontado de la sucursal asignada y la orden avanza a preparación/envío (PENDING_SHIPPING).
3. Dado que no hay stock suficiente de uno de los ítems en la sucursal seleccionada, cuando el cliente intenta confirmar el pedido, entonces el sistema bloquea el checkout y muestra el mensaje indicando el producto agotado.

b. Escenario de caso de uso:

Campo	Descripción
ID y Nombre	CU-01: Procesar orden de venta (Checkout)
Actores	Cliente (iniciador), Pasarela de pagos (MP/Efectivo), Administrador / Empleado (secundario)
Precondiciones	1. El cliente debe estar autenticado 2. El cliente debe tener al menos un producto en el carrito de compras (OrderStatus.ON_CART)
Trigger	El cliente hace click en el botón "Finalizar Compra" desde la vista del carrito.
Flujo principal	1. El cliente accede a la pantalla de checkout. 2. El sistema solicita seleccionar la sucursal (Office) y la dirección de entrega (Address). 3. El cliente entrega los datos y elige el medio de pago (mp, efectivo). 4. El cliente confirma la orden. 5. El sistema valida la existencia y stock de cada ítem en la sucursal elegida. 6. El sistema crea formalmente la SaleOrder vinculando cliente, items, dirección y sucursal con estado PENDING_PAYMENT. 7. El cliente completa el pago mediante la pasarela. 8. El sistema recibe la confirmación del pago, actualiza el estado de la orden a PAID, descuenta el stock correspondiente y dispara la generación de la factura.
Flujos alternativos / Excepciones	5a. Stock insuficiente: El sistema aborta la creación de la orden, notifica qué ítems no disponen de stock y mantiene la orden en estado ON_CART 7a. Rechazo o cancelación de pago: Si la pasarela rechaza la transacción o expira, la orden se mantiene en PENDING_PAYMENT o transición al CANCELLED. No se reserva ni descuenta stock definitivo. 8a. Cancelación posterior por el cliente / administrador: Si la orden se cancela (CANCELLED), el sistema incorpora el stock previamente descontado
Postcondiciones	La orden queda registrada con su UUID, estado, el

stock del inventario queda actualizado y se genera el comprobante fiscal.

Escenario: Generar Factura.



Ver la especificación detallada del [Escenario de caso de uso - Creación de factura](#) y su correspondiente [Diagrama de secuencia - Creación de factura](#).

a. Historia de usuario:

- i. ID: US-INV-01
- ii. Título: Emisión y consulta de facturas de compra
- iii. Como: Cliente o responsable de administración.
- iv. Quiero: Que se emita automáticamente un comprobante fiscal al confirmarse el pago de una orden y poder consultarlo en el sistema, para tener constancia contable y legal de la transacción comercial.

v. Criterios de aceptación:

1. Dado que una orden de venta pasa a estado PAID, cuando el servicio de pagos confirma la transacción, entonces se debe crear automáticamente una entidad Invoice con estado PAID, número de factura secuencial único y sus InvoiceDetail.
2. Dado que un usuario consulta el detalle de una orden pagada, cuando solicita ver su comprobante, entonces el sistema retorna la factura asociada con fecha, desglose de ítems, precios unitarios y monto total
3. Dado que una orden pagada es cancelada o reembolsada, cuando se ejecuta la cancelación, entonces el estado de la factura asociada debe actualizarse a CANCELLED

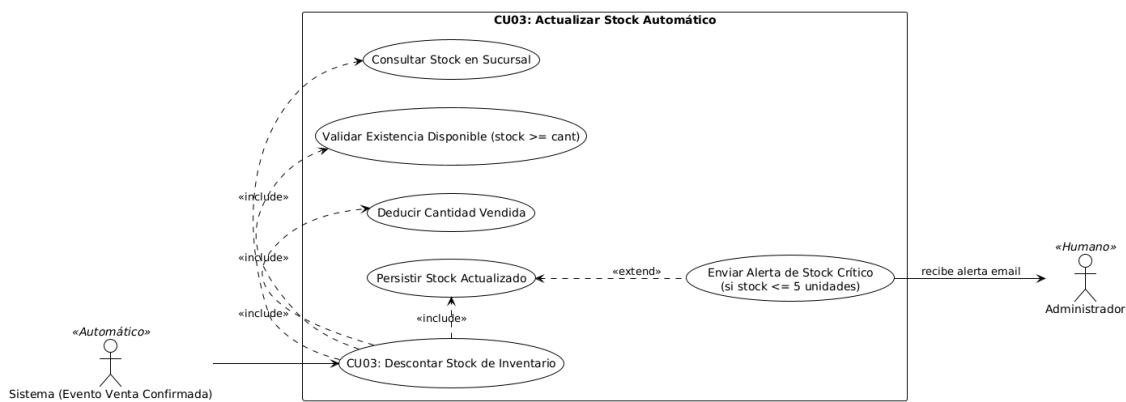
(InvoiceStatus.CANCELLED).

b. Escenario de caso de uso:

Campo	Descripción
ID y Nombre	CU-02: Generación y Gestión de Factura de Venta
Actores	Sistema ZeroShop (PaymentService, iniciador automático), Cliente (secundario), Administrador / Contador (secundario).
Precondiciones	1. Existe una SaleOrder con estado verificado PAID. 2. La orden debe contar con sus detalles de líneas (OrderDetail) con cantidad y precio unitario calculados.
Trigger	Confirmación exitosa del pago de la orden de venta
Flujo principal	1. El servicio de pagos invoca a createInvoiceForOrder(order, details). 2. El sistema verifica si ya existe una factura activa vinculada al orderId. 3. Al no existir, genera un número único correlativo de factura (ej. INV-YYYYMMDD-XXXX). 4. Se persiste la cabecera Invoice con: fecha actual, monto total (totalAmount), relación a la orden y estado InvoiceStatus.PAID. 5. Por cada OrderDetail del pedido, se crea un registro InvoiceDetail (descripción del producto, cantidad, precio unitario y subtotal). 6. El sistema guarda la factura y asocia los registros en la base de datos. 7. El cliente y el administrador pueden visualizar o descargar la factura desde su panel de pedidos.
Flujos alternativos / Excepciones	2a. La factura ya existía previamente: Si la factura existía en estado pendiente, el sistema actualiza su estado a PAID sin duplicar el número ni los registros.

	8a. Anulación de la orden: Ante una cancelación de pedido, se invoca <code>cancelInvoiceForOrder(orderId)</code> , marcando la factura como CANCELLED para fines de auditoría sin eliminar físicamente el registro (soft-delete o cambio de estado).
Postcondiciones	Comprobante fiscal registrado formalmente en la base de datos, vinculado de forma unívoca a la orden de venta correspondiente.

Escenario: Actualizar Stock.



Ver la especificación detallada del [Escenario de caso de uso - Modificación de stock](#) y su correspondiente [Diagrama de secuencia - Modificación de stock](#).

a. Historia de usuario:

- i. ID: US-STK-01
- ii. Título: Monitoreo, ajuste de inventario y notificación de stock crítico
- iii. Como: Administrador del inventario o encargado de sucursal, quiero consultar los niveles de stock por depósito, ajustar cantidades manualmente y recibir avisos automáticos cuando la existencia sea baja.
- iv. Para: Evitar quiebres de inventario y reabastecer a tiempo los productos de mayor rotación.

v. Criterios de aceptación:

1. Dado un producto en una sucursal específica, cuando el encargado realiza un ajuste de inventario (positivo o negativo), entonces la cantidad de stock se actualiza inmediatamente en el sistema.
2. Dado que la cantidad de stock de un producto desciende a 5 unidades o menos (`CRITICAL_STOCK_THRESHOLD <= 5`),

cuando se guarda la actualización, entonces el sistema envía automáticamente una alerta por correo electrónico a los usuarios con rol ADMIN.

3. Dado un intento de ajuste manual o descuento por venta, cuando la cantidad resultante fuera menor a 0, entonces el sistema rechaza la operación e informa que no es posible tener existencias negativas.

b. Escenario de caso de uso:

Campo	Descripción
ID y Nombre	CU-03: Control y Ajuste de Stock con Alerta Crítica
Actores	Administrador de Depósito / Empleado (iniciador), Servicio de Notificaciones (EmailService, sistema).
Precondiciones	1. El usuario debe haber iniciado sesión con rol de Administrador o Empleado con permisos de depósito. 2. El producto y la sucursal (Office) deben existir y estar activos en la plataforma.
Trigger	El usuario ingresa al panel de gestión de stock para auditar, recibir mercadería o corregir diferencias físicas.
Flujo principal	1. El usuario filtra el stock por producto o por sucursal (StockController). 2. El sistema muestra la lista de existencias (quantity) por cada combinación producto-sucursal. 3. El usuario selecciona un registro de stock e ingresa una modificación (ajuste relativo adjustStock o fijación absoluta updateStockQuantity). 4. El sistema valida que el valor final no sea negativo (quantity >= 0). 5. El sistema persiste el nuevo valor de quantity en la base de datos. 6. El sistema evalúa el umbral crítico: si quantity > 5, concluye la operación con mensaje de éxito. 7. El usuario visualiza el nuevo balance reflejado en el inventario.



Flujos alternativos / Excepciones	4a. Cantidad resultante negativa: El sistema arroja una excepción (IllegalStateException: "El ajuste resultaría en un stock negativo"), cancela la transacción y mantiene el valor previo. 6a. Stock crítico alcanzado (quantity <= 5): 6a.1. El sistema activa el método checkAndNotifyCriticalStock. 6a.2. Recupera todos los administradores activos (Role.ADMIN). 6a.3. Despacha correos electrónicos de alerta notificando el producto, código, sucursal y existencia actual. 6a.4. Se continúa con la confirmación de la operación.
Postcondiciones	El registro de stock queda actualizado en la base de datos y los administradores quedan informados en caso de requerir reabastecimiento.

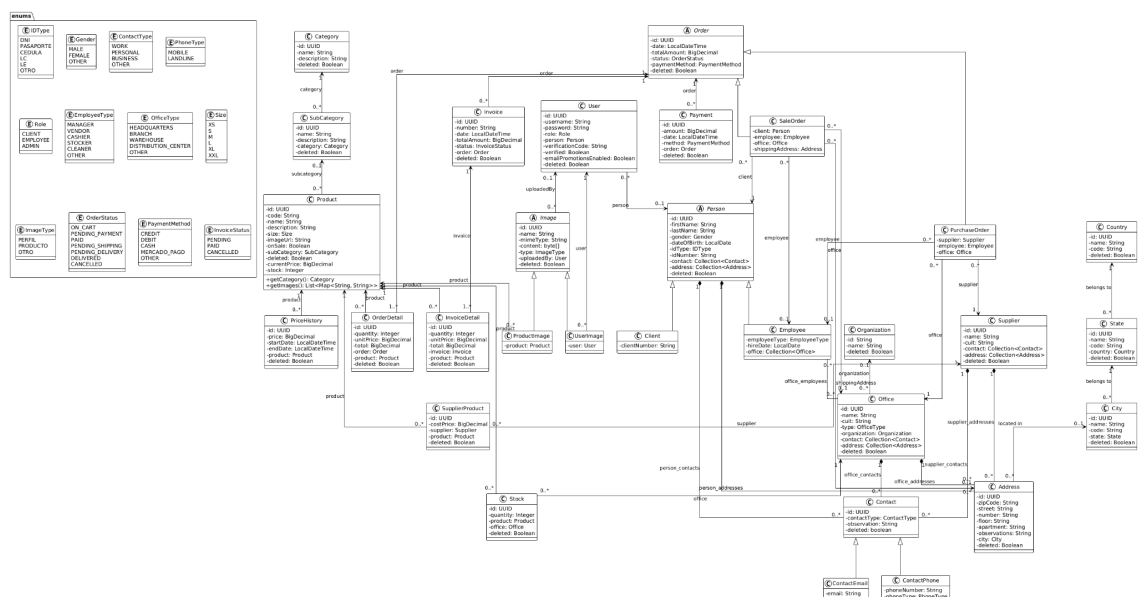
Diagrama de Secuencia del Sistema. Ver la especificación detallada de los diagramas de secuencia detallados en el Anexo.

Prototipado de pantalla de los casos de uso.

Análisis y Diseño.

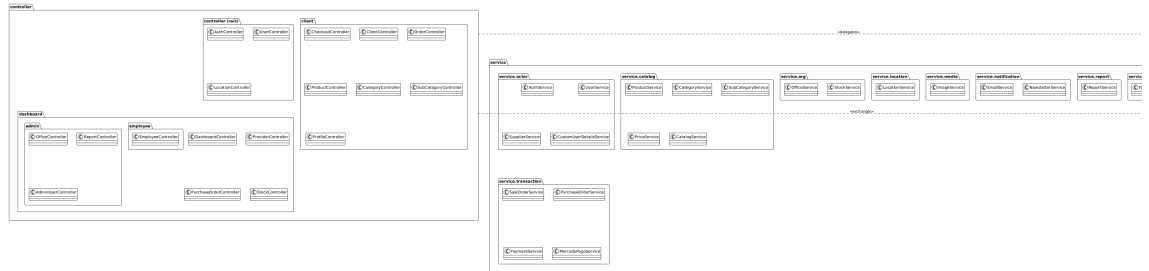
Se define la arquitectura base, el modelo de datos y las responsabilidades de cada componente del software.

- Diagrama de Clases de Diseño:



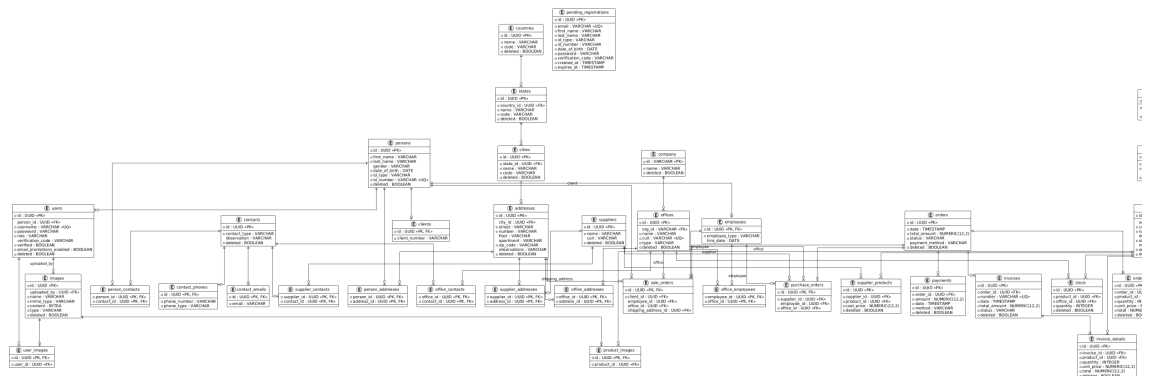
Ver el [Diagrama de clases](#) detallado en el Anexo.

- Diagramas de Secuencia de Diseño: Ver los diagramas de secuencia detallados en el Anexo.
- Diagrama de Paquetes de Diseño:



Ver el [Diagrama de paquetes](#) en el Anexo.

- Diagrama Entidad Relación Base de Datos:



Ver el [Diagrama entidad-relación](#) en el Anexo.

Patrones de Arquitectura y Diseño Aplicados.

Para garantizar la mantenibilidad y escalabilidad del código de Zero-Shop, el desarrollo fue guiado por los siguientes patrones de diseño, cumpliendo ampliamente con los requisitos técnicos del sistema:

1. **Modelo-Vista-Controlador (MVC):** Se utilizó Spring Web para los Controladores, Spring Data JPA para el Modelo y Thymeleaf para las Vistas (HTML dinámico en el navegador).
2. **Capas y Alta Cohesión / Bajo Acoplamiento:** El código se estructuró estrictamente en un diseño multicapa de /controller, /service, /repository. Lo que garantiza la alta cohesión (cada clase hace una sola cosa específica) y bajo acoplamiento (los

controladores no hablan directo con la base de datos, sino que pasan a través de los servicios).

3. Inyección de Dependencia: Para comunicar las capas entre sí, se utilizó el contenedor de Inyección de Dependencias de Spring. En lugar de crear objetos manualmente con new, los Repositorios se inyectan automáticamente en los Servicios, y los Servicios en los Controladores, facilitando además la ejecución de las pruebas unitarias.
4. DAO : Para el acceso a la base de datos no se utiliza JDBC crudo, sino el patrón DAO implementado mediante el ORM Hibernate y JPA. Se crearon Entidades con anotaciones JPA que mapean automáticamente las clases Java a las tablas relacionales de PostgreSQL.
5. DTO : Se implementó el patrón para el trasladar la información entre el frontend y el backend sin exponer la base de datos real. Por ejemplo, se utilizaron clases DTO como ClientProfileDTO y SalesReportDTO para agrupar los datos necesarios en la generación de los reportes administrativos de ventas y stock.
6. Experto en Información: La responsabilidad de aplicar la lógica compleja del negocio (cómo calcular el monto total de un carrito o realizar el descuento del inventario) no se dejó en los controladores, sino que se asignó a la capa de Servicios ya que son los componentes que poseen toda la información de las entidades involucradas.

2.d Fase de construcción.

En la presente iteración, el esfuerzo principal del ciclo RUP se focaliza en las tareas de codificación y verificación de componentes. De esta manera, las definiciones arquitectónicas consolidadas en la etapa anterior se traducen en artefactos ejecutables, materializando así la solución integral planificada.

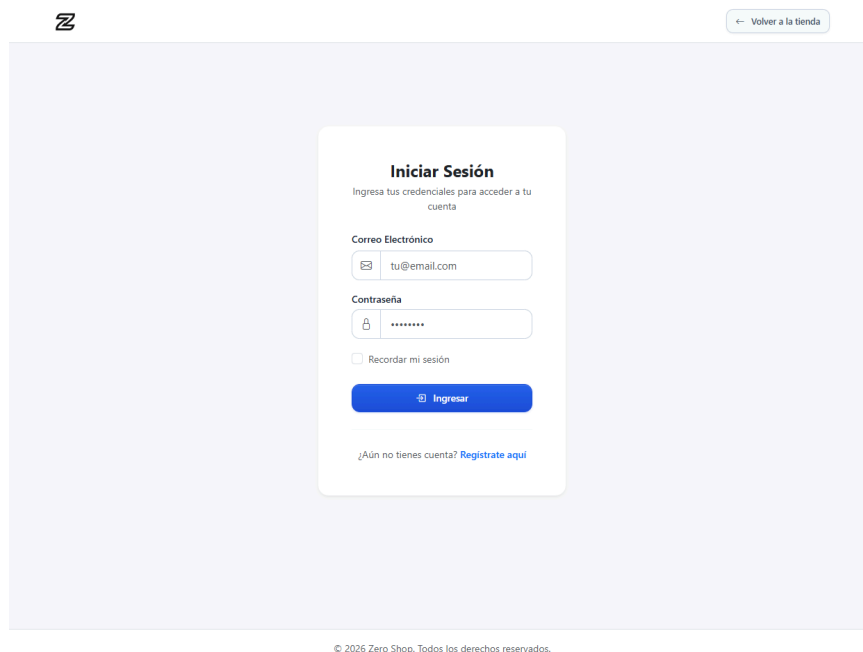
Flujo de Trabajo de Implementación.

A continuación, se exponen las especificaciones técnicas que fundamentan la construcción del sistema Zero-Shop:

- **Estructura y Tecnologías de Soporte:** El backend fue estructurado íntegramente utilizando Spring Boot. El patrón arquitectónico MVC se implementó mediante Spring Web, mientras que la capa visual dinámica se elaboró con plantillas Thymeleaf complementadas con el framework Bootstrap. Las tareas relativas a la autenticación, registro y encriptación fuertemente configurada de credenciales se delegaron en Spring Security. La gestión del código fuente y el control de versiones se llevaron a cabo centralizadamente a través de GitHub.
- **Persistencia de Datos:** El almacenamiento persistente se basa en la base de datos PostgreSQL. La interacción no se efectúa mediante sentencias SQL directas, sino a

través del ORM Hibernate apoyado en anotaciones de JPA, asegurando el mapeo fluido entre el modelo relacional y la representación orientada a objetos en Java.

- **Arquitectura del Código:** El software se organizó de forma estricta mediante una estructura multicapa articulada en cuatro paquetes para asegurar alta cohesión y bajo acoplamiento, incorporando además el patrón DTO:
 1. Controladores: Capturan las peticiones HTTP y administran el retorno hacia las interfaces visuales.
 2. Servicios: Encapsulan las reglas de negocio y la lógica operativa central del dominio.
 3. Repositorios: Gestionan la comunicación directa y las operaciones con la base de datos PostgreSQL.
 4. Objetos de Transferencia de Datos (DTO): Transportan la información necesaria entre las distintas capas, protegiendo la base de datos real.
- **Interfaz de Usuario y Prototipado:** Especificación y maquetación de los prototipos correspondientes al diagrama de navegación del sistema.
 1. Vista de autenticación de usuarios.



2. Panel de gestión para perfil administrativo.



Panel Staff Ver Tienda admin@gmail.com

Panel de Control
Resumen operativo general de la tienda y estado de almacén

VENTAS DEL MES
\$492.000,00
Período actual

ÓRDENES DEL MES
4
5 en gestión o pendientes

CATÁLOGO ACTIVO
12
Artículos registrados

ALERTAS DE STOCK
0
Inventario normal

Últimas Órdenes de Venta Ver Todas

Nº ORDEN	CLIENTE	TOTAL	ESTADO	ACCIÓN
#12afec9	Luciano Canovas	\$115.000,00	PAGO REALIZADO	
#6cdf835d	Luciano Canovas	\$115.000,00	PAGO REALIZADO	
#ed7b0fd1	Luciano Canovas	\$213.000,00	PAGO REALIZADO	
#3949be83	Luciano Canovas	\$49.000,00	ENTREGADO	
#874e26d5	Agustín Rodríguez	\$96.000,00	PAGO REALIZADO	

Alerta de Stock Crítico Ajustes

Todo el inventario activo se encuentra con niveles adecuados de stock.

3. Gestión de pedidos efectuados por clientes.

Panel Staff Ver Tienda admin@gmail.com

Órdenes de Venta de Clientes
Seguimiento de compras y despacho (PENDIENTE DE PAGO, PAGO REALIZADO, PENDIENTE DE ENVÍO, PENDIENTE DE ENTREGA, ENTREGADO)

Buscar por ID, Cliente o DNI Estado Sucursal

Nº pedido, nombre, DNIL... Todos los estados Todas las sucursales Filtrar X

Nº PEDIDO	FECHA	CLIENTE	TOTAL	MEDIO DE PAGO	ESTADO OFICIAL
#12afec9	01/10/2026 23:37	Luciano Canovas Sucursal: Casa Central Mendoza (Av. San Martín)	\$115000.00	Tarjeta de Crédito	PAGO REALIZADO
#6cdf835d	01/10/2026 23:34	Luciano Canovas Sucursal: Casa Central Mendoza (Av. San Martín)	\$115000.00	Efectivo	PAGO REALIZADO
#ed7b0fd1	01/10/2026 23:06	Luciano Canovas Sucursal: Casa Central Mendoza (Av. San Martín)	\$213000.00	Mercado Pago	PAGO REALIZADO
#3949be83	01/10/2026 22:34	Luciano Canovas Sucursal: Casa Central Mendoza (Av. San Martín)	\$49000.00	Mercado Pago	ENTREGADO
#874e26d5	29/09/2026 01:55	Agustín Rodríguez Sucursal: Sucursal Mendoza Plaza Shopping (Guaymallén)	\$96000.00	Mercado Pago	PAGO REALIZADO
#32372500	27/09/2026 01:55	Juan Pérez Sucursal: Sucursal Shopping Palmares (Godoy Cruz)	\$68000.00	Mercado Pago	PENDIENTE DE
#355e2d14	24/09/2026 01:55	Maria González Sucursal: Casa Central Mendoza (Av. San Martín)	\$113000.00	Tarjeta de Débito	ENTREGADO
#103d1352	18/09/2026 01:55	Juan Pérez Sucursal: Casa Central Mendoza (Av. San Martín)	\$137000.00	Tarjeta de Crédito	ENTREGADO

Mostrando 1 a 8 de 8 resultados

4. Gestión de órdenes dirigidas a proveedores.



Órdenes de Compra				
Reabastecimiento de mercadería con proveedores e ingreso automático a inventario				
Buscar por ID o Proveedor: N° o Proveedor... Proveedor: Todos los proveedores Sucursal: Todas las sucursales Estado: Todos los estados Filtrar				
N° PEDIDO	FECHA	PROVEEDOR	SUCURSAL DESTINO	COSTO TOTAL
#OC-BA347FBC	28/09/2026 01:55	Accesorios Urbanos y Moda S.A.	Sucursal Mendoza Plaza Shopping (Guaymallén)	\$1.075.000,00
#OC-4E2FB29D	23/09/2026 01:55	Calzados y Deportes del Plata S.R.L.	Sucursal Shopping Palmares (Godoy Cruz)	\$3.260.000,00
#OC-245A60AB	12/09/2026 01:55	Distribuidora Textil Cuyana S.A.	Casa Central Mendoza (Av. San Martín)	\$3.300.000,00

Mostrando 1 a 3 de 3 resultados

Testing (Pruebas unitarias y de interfaz).

La verificación y validación del sistema se sustentaron en pruebas unitarias e integrales desarrolladas con JUnit y Mockito, posibilitando la comprobación continua a medida que avanzaba el proyecto. Este procedimiento aseguró el correcto desempeño de procesos complejos, tales como la integración con Mercado Pago, el procesamiento de ventas y la actualización de carritos de compra.

Adicionalmente, se incorporaron herramientas fundamentadas en Inteligencia Artificial mediante Playwright con el propósito de examinar de forma automatizada los componentes dinámicos de la interfaz, abarcando botones, transiciones y elementos interactivos del frontend.

Complementariamente, el equipo de desarrollo llevó a cabo instancias de pruebas manuales sobre las pantallas para verificar que la navegación y el comportamiento general del aplicativo respondieron de manera adecuada a las especificaciones predefinidas.

Gestión del Cambio

La administración y trazabilidad del código se articularon mediante la plataforma GitHub dentro de un repositorio centralizado, posibilitando el trabajo colaborativo de los integrantes del proyecto. Durante el avance del desarrollo surgieron diversos ajustes operacionales, debido a que el abordaje práctico de los casos de uso derivó en la identificación de alternativas técnicas más eficientes que aquellas planteadas en los modelos de clases iniciales.

2.e Fase de transición y despliegue.

En ésta etapa final del ciclo, el objetivo es transicionar el producto de software terminado desde el entorno de desarrollo hacia el usuario final, asegurando que esté listo para operar en condiciones reales. El equipo de desarrollo se enfocó en empaquetar el producto final y prepararlo para su entrega y revisión por parte de los evaluadores, aunque, no se pudo testear en un entorno de producción de modo que se realice por completo la etapa del despliegue.

3. Propuesta de mejora para software de gimnasios.

3.a Referencias de mercado

Las distintas alternativas de software disponibles en el mercado destinadas al control de centros deportivos brindan soluciones globales para la administración de usuarios y membresías. Estas plataformas integran herramientas avanzadas como validación biométrica mediante huella o reconocimiento facial, aplicaciones móviles dedicadas, métricas detalladas y la automatización de notificaciones vía correo electrónico o WhatsApp.

[ABC Evo](#)

Ecosistema informático para la administración completa que dispone de soporte biométrico facial, aplicación exclusiva para miembros, módulo CRM conectado a WhatsApp, analítica de datos e integración directa con más de 50 dispositivos de acceso físico.

[GymFlow](#)

Herramienta orientada al contexto nacional, diseñada para el monitoreo instantáneo de pagos pendientes, administración del flujo de caja, avisos automatizados de cuotas expiradas y armado de rutinas sincronizado con monitores en sala.

[AgendaPRO](#)

Plataforma de amplio uso en el ámbito local para la organización de turnos, envío de avisos automáticos, procesamiento de cobros y liquidación de comisiones para instructores desde un único panel centralizado.

3.b Propuestas de mejora

La mejora propuesta por el grupo consiste en la implementación de un módulo de gestión personalizada del entrenamiento, donde los clientes puedan definir objetivos específicos en conjunto con un entrenador asignado por el gimnasio. A continuación se detallan los componentes principales de esta propuesta:

Asignación y Seguimiento por Entrenador Personalizado

- Asignación automática o manual de un entrenador según el perfil, disponibilidad y objetivos del cliente.
- Canal de comunicación directo y feedback continuo dentro de la plataforma para ajustar el plan según el progreso.

Planes y Rutinas de Entrenamiento Personalizados

- Diseño de rutinas adaptadas a las metas individuales (hipertrofia, pérdida de peso, rendimiento deportivo, rehabilitación).
- Registro detallado de series, repeticiones, cargas y tiempos de descanso con seguimiento del avance histórico.

Orientación Nutricional y Guías de Alimentación

- Elaboración de pautas nutricionales y planes de alimentación complementarios ajustados al gasto calórico y objetivos del socio.
- Recomendaciones de hidratación, suplementación y hábitos saludables.

Consejos y Recomendaciones Técnicas (Tips)

- Guías multimedia sobre correcta ejecución postural para prevenir lesiones durante los ejercicios.

Tips diarios sobre recuperación muscular, manejo del descanso y motivación deportiva.

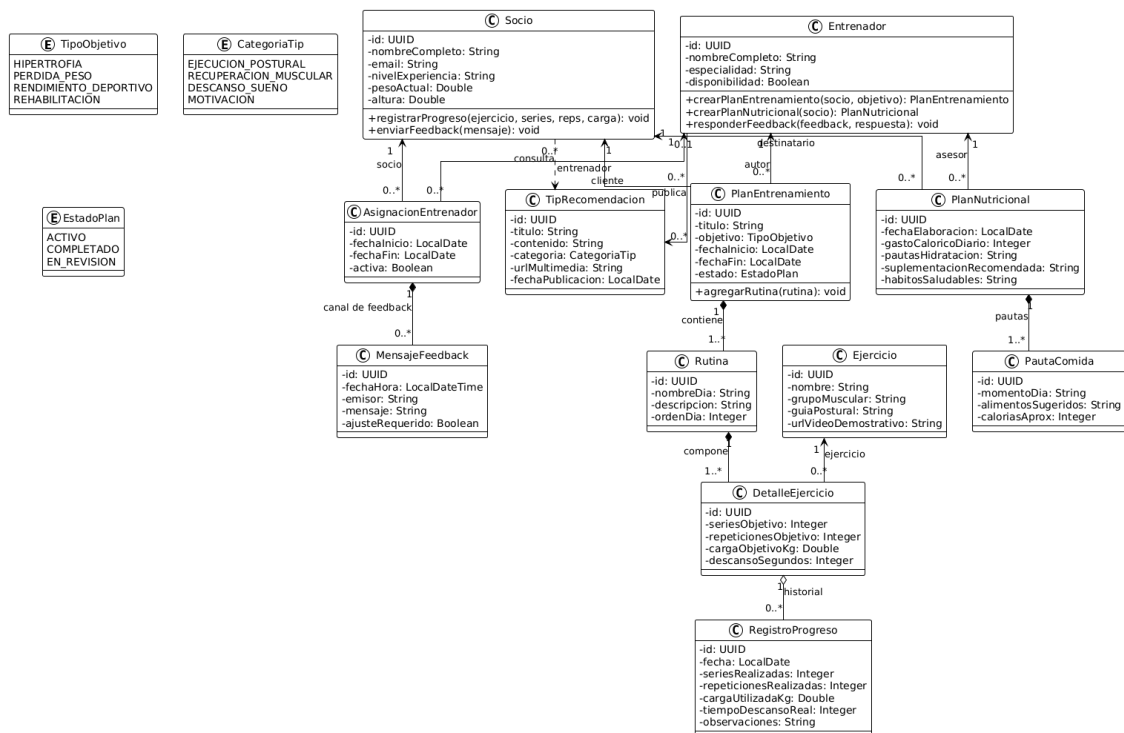


Diagrama de clases de diseño - Propuesta de mejora GYM



4. Propuestas de refactorización

4.a Propuestas de refactorización de la vista

En lo respectivo a la capa de presentación, el equipo plantea que la sustitución de las tecnologías Thymeleaf y Bootstrap por una arquitectura del tipo Single Page Application (SPA) basada en React resultaría más beneficiosa para el proyecto. Dicha alternativa, fundamentada en la reutilización de componentes, incrementa la eficiencia general del desarrollo y otorgaría un dinamismo superior a la experiencia del usuario.

4.b Propuestas de refactorización de los controladores

En cuanto a la refactorización de la capa de controladores, se propone la consolidación de las operaciones redundantes mediante la definición de un controlador genérico. Actualmente, diversos componentes ejecutan tareas análogas —particularmente en las gestiones de ABM—, por lo que su unificación permitiría mitigar la duplicación de código y optimizar la mantenibilidad general del sistema.

4.c Propuestas de refactorización del modelo

Con el propósito de optimizar la capa de servicio del modelo, se sugiere la creación de un único componente denominado CatalogService para la administración del catálogo comercial. La problemática detectada radica en la fragmentación excesiva producida por la coexistencia de múltiples servicios, lo que ocasiona acoplamiento innecesario en los controladores (como ocurre en ProductController, el cual requiere inyectar tanto ProductService como CatalogService). Mediante este nuevo servicio centralizado, se concentraría la gestión integral de productos, precios y categorías.

4.d Propuestas de refactorización del acceso a datos

Tras concluir la etapa de desarrollo, el equipo determinó que la estructuración del paquete de acceso a datos evidencia oportunidades de mejora, principalmente asociadas con la sobre-fragmentación de interfaces. En la arquitectura actual de repositorios se identifican entidades fuertemente dependientes separadas en distintos archivos, tales como OrderRepository junto a OrderDetailRepository, o bien InvoiceRepository e InvoiceDetailRepository. Al carecer de operatividad autónoma, resulta conveniente unificar estas interfaces secundarias en sus componentes principales correspondientes. Del mismo modo, para el módulo Location, se plantea integrar las cuatro interfaces existentes en una única interfaz denominada LocationRepository.

Esta reestructuración mantiene los principios de alta cohesión y bajo acoplamiento, removiendo el ruido arquitectónico derivado de una descomposición innecesaria y favoreciendo la mantenibilidad futura.

5. Material de exposición en conferencia

 Presentación Zero Shop - IS2

[Acceso al repositorio del proyecto](#)

Credenciales de prueba para la plataforma Mercado Pago:

- **Usuario:**
TESTUSER1804946326069841117
- **Contraseña:**
R5VnOtvgyC

6. Referencias

- Material bibliográfico y diapositivas de la cátedra.
- <https://plantuml.com/>
- <https://agendapro.com/ar/gimnasio/software-para-gimnasio>
- <https://www.gymflow.com.ar/>
- <https://softwareparagimnasio.com/>

7. Anexo

- Especificación del [Diagrama de clases](#).
- Especificación del [Diagrama de paquetes](#).
- Especificación del [Diagrama entidad-relación](#).
- Especificación del [Diagrama de caso de uso - Casos de uso críticos](#).
- Caso de uso: Orden de compra cliente:
 - Especificación detallada del [Escenario de caso de uso - Orden de compra cliente](#).
 - Especificación del [Diagrama de secuencia - Orden de compra de cliente](#).
- Caso de uso: Creación de factura:
 - Especificación detallada del [Escenario de caso de uso - Creación de factura](#).
 - Especificación del [Diagrama de secuencia - Creación de factura](#).
- Caso de uso: Modificación de stock:
 - Especificación detallada del [Escenario de caso de uso - Modificación de stock](#).
 - Especificación del [Diagrama de secuencia - Modificación de stock](#).